

Codefest 6

Hardware for Artificial Intelligence and Machine Learning (HW4AI)

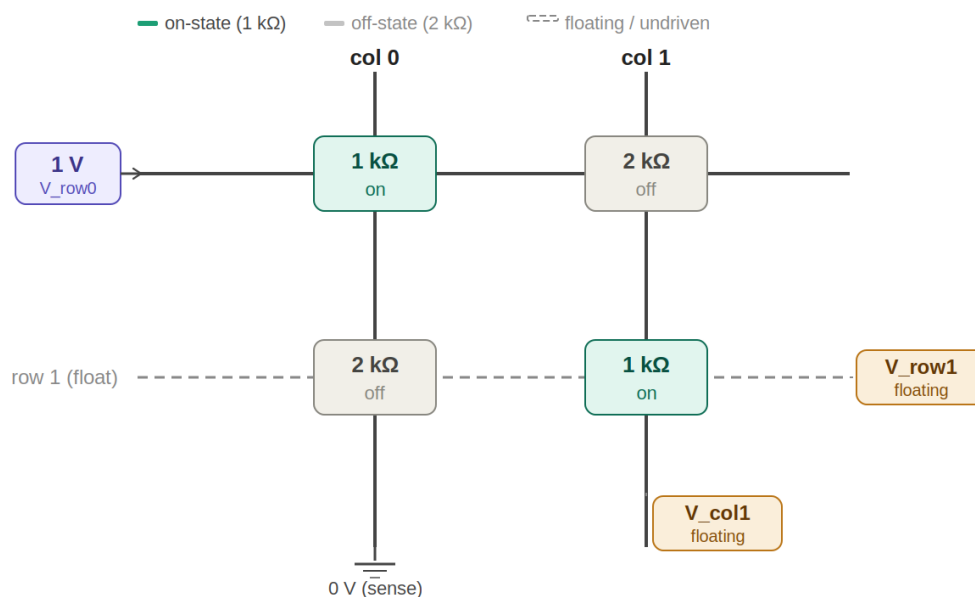
ECE 410/510

Spring 2026

CMAN — Sneak paths in a resistive crossbar

Consider the 2×2 resistive crossbar below. Rows carry input voltages; columns carry output currents. Cell resistances are fixed: $R[0][0] = 1 \text{ k}\Omega$, $R[0][1] = 2 \text{ k}\Omega$, $R[1][0] = 2 \text{ k}\Omega$, $R[1][1] = 1 \text{ k}\Omega$. The low-resistance cells ($1 \text{ k}\Omega$) encode “on” weights; the high-resistance cells ($2 \text{ k}\Omega$) encode “off” weights.

2×2 resistive crossbar — circuit setup



Tasks

1. Ideal read. Apply $V_{\text{row0}} = 1 \text{ V}$ to row 0. Assume col 0 is held at virtual ground (0 V) for current sensing, and that row 1 and col 1 are also grounded. Compute the ideal output current I_{col0} .
2. Sneak path read. Now apply $V_{\text{row0}} = 1 \text{ V}$ to row 0 and hold col 0 at 0 V , but leave row 1 and col 1 floating (undriven). Use *Kirchhoff's Current Law* (KCL) to find the floating node voltages V_{row1} and V_{col1} , then compute the actual current I_{col0} including the sneak path contribution.
3. In 2–3 sentences, explain why the sneak path current corrupts the intended *Matrix Vector Multiplication* (MVM) result and what it implies for reading large crossbar arrays.

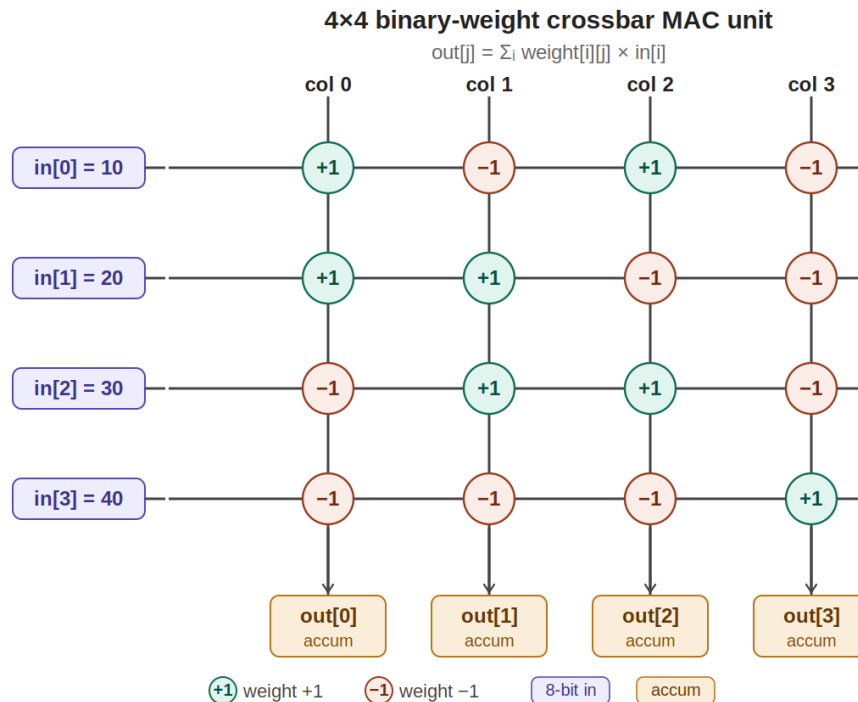
CMAN NO AI

Deliverable:

- codefest/cf06/cman_sneak_path.md (or .pdf/.png) containing
 - (a) ideal I_{col0} with calculation;
 - (b) KCL solution for floating node voltages V_{row1} and V_{col1} ;
 - (c) actual I_{col0} with sneak path current itemized;
 - (d) 2–3 sentence explanation of how sneak paths corrupt MVM results.

CLLM — 4×4 crossbar MAC unit in SystemVerilog

Use an LLM to generate a SystemVerilog module for a 4×4 binary-weight crossbar MAC unit: 4 input lines (8-bit signed), 4 output lines (accumulator), weights encoded as +1/-1 stored in a 4×4 register array. Each clock cycle computes $out[j] = \sum_i weight[i][j] \times in[i]$. Write a testbench, simulate, and present your work in Shine.



Tasks

4. Use an LLM to generate the crossbar MAC module. Save as `codefest/cf06/hdl/crossbar_mac.sv`.
5. Write a testbench `codefest/cf06/hdl/crossbar_tb.sv`. Load weights `[[1,-1,1,-1],[1,1,-1,-1],[-1,1,1,-1],[-1,-1,-1,1]]`, apply input `[10, 20, 30, 40]`, compute the expected outputs by hand, simulate, and confirm the results match.
6. Record a 5-minute Shine presentation explaining:
 - (a) the module design and how the crossbar computes $out[j]$;
 - (b) your testbench strategy and the input/weight values used;
 - (c) your simulation results and whether they matched the hand-calculated outputs.

CLLM AI OK

Deliverable: GitHub commit containing

- `codefest/cf06/hdl/crossbar_mac.sv` (LLM-generated);
- `codefest/cf06/hdl/crossbar_tb.sv`;
- simulation log showing correct output.
- Shine presentation (≈5 min) explaining the design, testbench, and simulation results.